WeatherWise

> **The weather app that tells you when to trust the forecast.**

WeatherWise pulls forecasts from multiple weather APIs, computes a weighted consensus reading, and surfaces a **Forecast Dispute** indicator when sources strongly disagree — so you know exactly how much confidence to place in the forecast.

Features

- **Weighted Consensus** — aggregates multiple weather sources into a single reading, weighted by source accuracy (equal weighting in Phase 1, adaptive in Phase 3)
- **Forecast Dispute Indicator** — detects when sources spread >3 °C and shows an amber/red badge with plain-English explanation
- **Confidence Bar** — 0–100% visual agreement meter ("Strong Agreement" / "Moderate" / "Disputed")
- **Source Breakdown Panel** — toggle to see each source's reading side-by-side, with the outlier highlighted
- **7-Day Forecast Chart** — high/low trend lines with per-day spread bands and precipitation overlay
- **°C / °F Toggle** — convert at display time; all internals use Celsius
- **In-memory cache** — 10-minute TTL per city to stay within free-tier rate limits
- **Graceful degradation** — if one source fails, the others still contribute

---

Tech Stack

Layer	Technology
Frontend	React 18 + Vite, TypeScript
Styling	Tailwind CSS
Charts	Recharts
Data fetching	TanStack Query (React Query v5)
Routing	React Router v6
Backend	Node.js + Express, TypeScript
HTTP client	Axios
Cache	node-cache (in-memory)
Dev tooling	tsx, concurrently, Vitest

Project Structure

```
weatherwise/
├── client/                        # React + Vite frontend
│   ├── src/
│   │   ├── components/
│   │   │   ├── ConsensusCard.tsx   # Hero temp + condition
│   │   │   ├── ConfidenceBar.tsx   # Agreement meter
│   │   │   ├── DisputeBadge.tsx    # High uncertainty indicator
│   │   │   ├── WeatherStats.tsx    # Feels like, humidity, wind, precip
│   │   │   ├── SourceBreakdown.tsx # Side-by-side source table
│   │   │   ├── ForecastChart.tsx   # 7-day chart + day strip
│   │   │   └── SearchBar.tsx       # City search form
│   │   ├── hooks/
│   │   │   └── useWeatherConsensus.ts  # React Query hooks
│   │   ├── utils/
│   │   │   └── formatters.ts       # Temp units, dates, emojis
│   │   ├── types/
│   │   │   └── weather.ts          # Shared TypeScript interfaces
│   │   ├── App.tsx                 # Main dashboard
│   │   ├── main.tsx
│   │   └── index.css
│   ├── index.html
│   ├── vite.config.ts             # Dev proxy → localhost:3001
│   ├── tailwind.config.js
│   └── package.json
│
├── server/                        # Express backend
│   ├── src/
│   │   ├── routes/
│   │   │   └── weather.ts          # GET /api/weather/current|forecast|sources
│   │   ├── services/
│   │   │   ├── openMeteo.ts        # Open-Meteo adapter (no key required)
│   │   │   ├── openWeatherMap.ts   # OpenWeatherMap adapter
│   │   │   └── consensus.ts        # Aggregation + dispute logic
│   │   ├── cache/
│   │   │   └── weatherCache.ts     # node-cache wrapper
│   │   ├── types/
│   │   │   └── weather.ts          # Server-side type definitions
│   │   ├── __tests__/
│   │   │   └── consensus.test.ts   # Unit tests (Vitest)
│   │   └── index.ts               # Express app entry point
│   ├── tsconfig.json
│   └── package.json
│
├── .env.example                   # Environment variable template
├── package.json                   # Workspace root + concurrently
└── README.md
```

Getting Started

Prerequisites

- **Node.js v18+** (v22 recommended)
- **npm v9+**
- An **OpenWeatherMap API key** (free at openweathermap.org) — optional; Open-Meteo works without any key

1. Clone and install

```
git clone https://github.com/alikhademisullivan/weatherwise.git
cd weatherwise
npm install
```

2. Configure environment

```
cp .env.example .env
```

Edit `.env`:

```
PORT=3001
OPENWEATHERMAP_API_KEY=your_key_here   # optional but recommended
CACHE_TTL_SECONDS=600                  # 10 minutes
DISPUTE_THRESHOLD_CELSIUS=3            # spread > 3°C triggers dispute badge
```

> Open-Meteo requires **no API key** and is always used. OpenWeatherMap is used automatically when `OPENWEATHERMAP_API_KEY` is set.

3. Run in development

```
npm run dev
```

This starts both servers concurrently:

- **Backend** → `http://localhost:3001`
- **Frontend** → `http://localhost:5173` (proxies `/api` to the backend)

Open `http://localhost:5173` in your browser.

API Reference

`GET /api/weather/current?city=<city>`

Returns the consensus reading and per-source breakdown for the current conditions.

```
{
  "location": "Toronto",
  "consensus": {
    "temperature": 18.2,
    "feelsLike": 16.8,
    "humidity": 62,
    "windSpeed": 14.3,
    "precipitationProbability": 25,
    "condition": "Partly Cloudy",
    "sources": [...],
    "spread": 2.1,
    "confidenceScore": 86,
    "isDisputed": false,
    "disputeMessage": "Sources are in good agreement on today's forecast.",
    "location": "Toronto",
    "updatedAt": "2025-06-11T14:00:00.000Z"
  },
  "sources": [
    {
      "source": "Open-Meteo",
      "temperature": 17.4,
      "feelsLike": 15.9,
      "humidity": 65,
      "windSpeed": 12.0,
      "precipitationProbability": 20,
      "condition": "Partly Cloudy",
      "conditionCode": "partly_cloudy",
      "fetchedAt": "2025-06-11T14:00:00.000Z"
    }
  ],
  "updatedAt": "2025-06-11T14:00:00.000Z"
}
```


`GET /api/weather/forecast?city=<city>&days=7`

Returns a merged 7-day forecast with spread bands from all sources.

`GET /api/weather/sources?city=<city>`

Returns raw per-source readings without consensus computation.

`GET /api/health`

Health check: `{ "status": "ok", "timestamp": "..." }`

Consensus & Dispute Logic

How consensus is computed

Each source returns a `SourceReading` with normalized fields (Celsius, km/h, etc.). The consensus layer:

1. **Weighted average** — each numeric field (temp, humidity, wind, precip) is averaged using source weights (currently 1.0 for all, tunable in Phase 3)
2. **Majority-vote condition** — the most-weighted condition string wins ("Rain", "Partly Cloudy", etc.)
3. **Spread** — `max(temps) - min(temps)` across all sources
4. **Confidence score** — `max(0, 100 - (spread / 10) * 100)`

Dispute thresholds

Condition	Badge	Message
Spread > 5°C	🔴 High Uncertainty	"Sources strongly disagree — treat this forecast with caution."
Spread > 3°C	⚠️ Forecast Disputed	"Sources are split on temperature — pack layers just in case."
Precip spread > 30%	⚠️ Forecast Disputed	"Sources can't agree on rain chances — bring an umbrella."
No dispute	✓ (green)	"Sources are in good agreement on today's forecast."

The threshold is configurable via `DISPUTE_THRESHOLD_CELSIUS` in `.env`.

Running Tests

```
# All tests
npm test

# Server tests only
npm test --workspace=server

# Client tests only
npm test --workspace=client
```

Build for Production

```
npm run build
```

- **Server** → `server/dist/` (compiled JS, run with `node dist/index.js`)
- **Client** → `client/dist/` (static assets, serve with any static host or nginx)

To run the built server:

```
cd server && node dist/index.js
```

Weather Sources

Source	Key Required	Coverage	Notes
Open-Meteo	No	Global	Geocoding via their free geocoding API
OpenWeatherMap	Yes (free tier)	Global	1,000 calls/day free
Tomorrow.io	Yes	Global	Phase 2 — probabilistic forecasts
WeatherAPI.com	Yes	Global	Phase 2 — air quality, astronomy

Development Phases

Phase 1 — MVP (current)

- [x] Express server with Open-Meteo + OpenWeatherMap adapters
- [x] Consensus and dispute logic
- [x] In-memory cache
- [x] React frontend: dashboard, confidence bar, dispute badge, source breakdown
- [x] 7-day forecast with spread visualization
- [x] City search
- [x] Unit tests for consensus logic

Phase 2 — Polish

- [ ] Tomorrow.io and WeatherAPI.com adapters
- [ ] Geolocation (`navigator.geolocation`)
- [ ] Hourly forecast view
- [ ] Animated weather condition backgrounds

Phase 3 — Smart Weighting

- [ ] PostgreSQL: store predictions vs. actual outcomes per source
- [ ] Rolling accuracy score per source per region
- [ ] Feed accuracy scores into weighted averages dynamically

Environment Variables

Variable	Default	Description
`PORT`	`3001`	Express server port
`OPENWEATHERMAP_API_KEY`	—	OWM API key (free tier: 1k calls/day)
`TOMORROW_IO_API_KEY`	—	Tomorrow.io API key (Phase 2)
`WEATHERAPI_KEY`	—	WeatherAPI.com key (Phase 2)
`CACHE_TTL_SECONDS`	`600`	Cache lifetime per city in seconds

Variable	Default	Description
`DISPUTE_THRESHOLD_CELSIUS`	`3`	Spread (°C) that triggers the dispute badge

Contributing

1. Fork the repo and create your branch from `main`
2. Run `npm install` at the repo root
3. Make your changes with TypeScript throughout
4. Run `npm test` to verify all tests pass
5. Submit a pull request with a clear description